

Coches eléctricos

Skarleth Motino Flores

20 de Abril de 2023

Introducción

En este proyecto se desarrolla en Python un análisis básico de datos sobre Vehículos Eléctricos vendidos en los Estados Unidos. Los datos provienen de <https://data.wa.gov/Transportation/Electric-Vehicle-Population-Data/f6w7-q2d2> (<https://data.wa.gov/Transportation/Electric-Vehicle-Population-Data/f6w7-q2d2>). A partir de ellos he realizado varias exploraciones y visualizaciones de autos vendidos por estado, tecnologías o marcas. He conseguido todos los problemas planteados, y en el problema libre, por falta de tiempo he propuesto un problema sencillo de visualización usando la librería de Geopandas.

Librerías

Pongamos todas las librerías necesarias al principio, tal como propone el estilo `pep-8`. Ej.: [PEP 8 -- Style Guide for Python Code](https://www.python.org/dev/peps/pep-0008/) (<https://www.python.org/dev/peps/pep-0008/>).

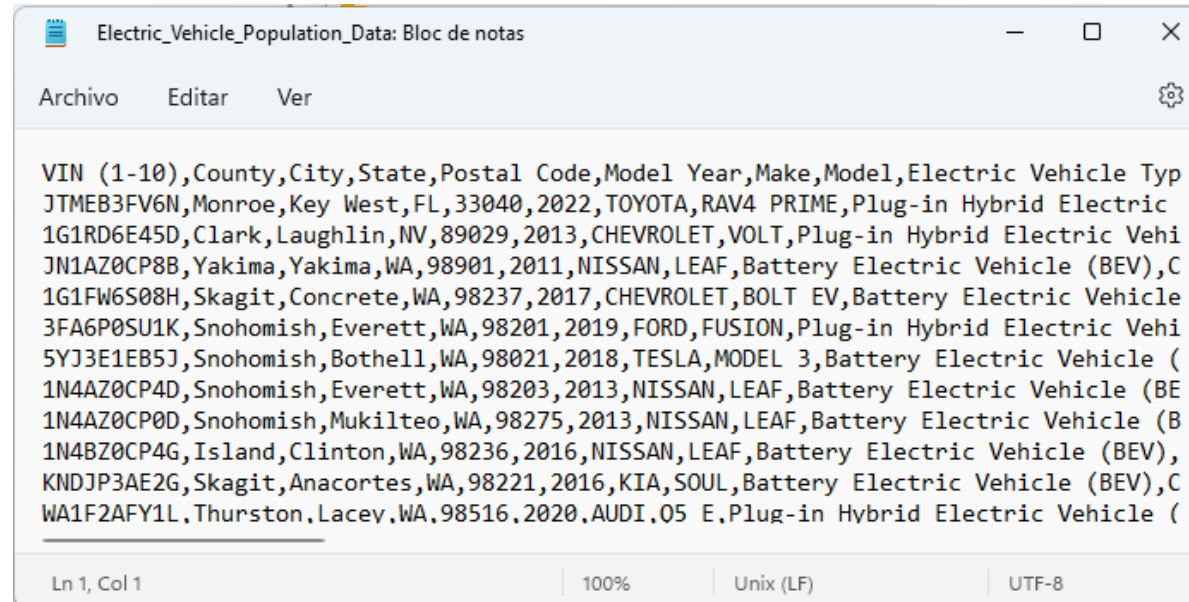
```
In [8]: 1 # Esta celda debe ser completada por el estudiante.
2
3 import pandas as pd
4 import numpy as np
5 import matplotlib.pyplot as plt
6 import geopandas
```

a) Algunas operaciones sencillas [3 puntos]

Vamos a trabajar con una tabla de datos que contiene información sobre vehículos eléctricos y vehículos híbridos enchufables registrados en el Departamento de Licencias de Washington. Mi última actualización es de 9 de diciembre de 2022. La URL de referencia es la siguiente: <https://data.wa.gov/Transportation/Electric-Vehicle-Population-Data/f6w7-q2d2> (<https://data.wa.gov/Transportation/Electric-Vehicle-Population-Data/f6w7-q2d2>).

En ella puede encontrarse información más detallada, así como una descripción precisa de cada columna.

Nuestra tabla de datos es un archivo de texto (`Electric_Vehicle_Population_Data.csv`) que puede verse así con cualquier editor:



```
Electric_Vehicle_Population_Data: Bloc de notas
Archivo  Editar  Ver

VIN (1-10),County,City,State,Postal Code,Model Year,Make,Model,Electric Vehicle Typ
JTMEB3FV6N,Monroe,Key West,FL,33040,2022,TOYOTA,RAV4 PRIME,Plug-in Hybrid Electric
1G1RD6E45D,Clark,Laughlin,NV,89029,2013,CHEVROLET,VOLT,Plug-in Hybrid Electric Vehi
JN1AZ0CP8B,Yakima,Yakima,WA,98901,2011,NISSAN,LEAF,Battery Electric Vehicle (BEV),C
1G1FW6S08H,Skagit,Concrete,WA,98237,2017,CHEVROLET,BOLT EV,Battery Electric Vehicle
3FA6P0SU1K,Snohomish,Everett,WA,98201,2019,FORD,FUSION,Plug-in Hybrid Electric Vehi
5YJ3E1EB5J,Snohomish,Bothell,WA,98021,2018,TESLA,MODEL 3,Battery Electric Vehicle (
1N4AZ0CP4D,Snohomish,Everett,WA,98203,2013,NISSAN,LEAF,Battery Electric Vehicle (BE
1N4AZ0CP0D,Snohomish,Mukilteo,WA,98275,2013,NISSAN,LEAF,Battery Electric Vehicle (B
1N4BZ0CP4G,Island,Clinton,WA,98236,2016,NISSAN,LEAF,Battery Electric Vehicle (BEV),
KNDJP3AE2G,Skagit,Anacortes,WA,98221,2016,KIA,SOUL,Battery Electric Vehicle (BEV),C
WA1F2AFY1L,Thurston,Lacey,WA,98516,2020,AUDI,05 E,Plug-in Hybrid Electric Vehicle (

Ln 1, Col 1 | 100% | Unix (LF) | UTF-8
```

La primera columna es la cabecera, y contiene los nombres de los campos, separados por comas. Las demás, son los valores de dichos campos, consignando los datos de cada vehículo en una línea.

Si la abrimos con excell, vemos cada línea en una celda, sin separar los distintos campos:

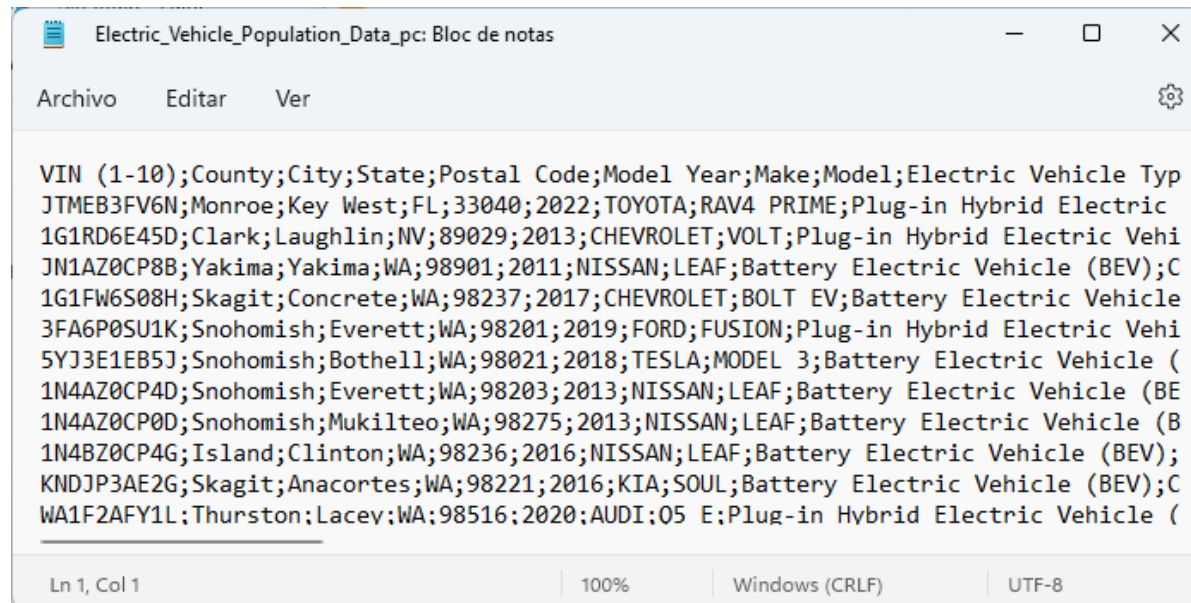
	A	B	C	D	E	F	G	H	I	J
1	VIN (1-10),County,City,State,Postal Code,Model Year,Make,Model,Electric Vehicle Type,Clean Alternative Fuel Vehicle (CAFV)									
2	JTMEB3FV6N,Monroe,Key West,FL,33040,2022,TOYOTA,RAV4 PRIME,Plug-in Hybrid Electric Vehicle (PHEV),Clean Alternative Fu									
3	1G1RD6E45D,Clark,Laughlin,NV,89029,2013,CHEVROLET,VOLT,Plug-in Hybrid Electric Vehicle (PHEV),Clean Alternative Fuel Veh									
4	JN1AZ0CP8B,Yakima,Yakima,WA,98901,2011,NISSAN,LEAF,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eligible									
5	1G1FW6S08H,Skagit,Concrete,WA,98237,2017,CHEVROLET,BOLT EV,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle									
6	3FA6P0SU1K,Snohomish,Everett,WA,98201,2019,FORD,FUSION,Plug-in Hybrid Electric Vehicle (PHEV),Not eligible due to low b									
7	5YJ3E1EB5J,Snohomish,Bothell,WA,98021,2018,TESLA,MODEL 3,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eli									
8	1N4AZ0CP4D,Snohomish,Everett,WA,98203,2013,NISSAN,LEAF,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eli									
9	1N4AZ0CP0D,Snohomish,Mukilteo,WA,98275,2013,NISSAN,LEAF,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle E									
10	1N4BZ0CP4G,Island,Clinton,WA,98236,2016,NISSAN,LEAF,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eligible,									
11	KNDJP3AE2G,Skagit,Anacortes,WA,98221,2016,KIA,SOUL,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eligible,S									
12	WA1F2AFY1L,Thurston,Lacey,WA,98516,2020,AUDI,Q5 E,Plug-in Hybrid Electric Vehicle (PHEV),Not eligible due to low battery r									
13	5YJXCBE23L,Grant,Moses Lake,WA,98837,2020,TESLA,MODEL X,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eli									
14	3C3CFFGE3G,St. Clair,Mascoutah,IL,62258,2016,FIAT,500,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eligible,84									

a.1) Cambiar el formato del archivo csv a "punto y coma"

Podemos importar la tabla de datos desde excell (pestaña datos), simplemente indicando que el separador es una coma:

	A	B	C	D	E	F	G	H	I	J	K	L
1	VIN (1-10)	County	City	State	Postal Code	Model Year	Make	Model	Electric Vehi	Clean Altern	Electric Rang	Base MSRP
2	JTMEB3FV6N	Monroe	Key West	FL	33040	2022	TOYOTA	RAV4 PRIME	Plug-in Hybr	Clean Altern	42	0
3	1G1RD6E45D	Clark	Laughlin	NV	89029	2013	CHEVROLET	VOLT	Plug-in Hybr	Clean Altern	38	0
4	JN1AZ0CP8B	Yakima	Yakima	WA	98901	2011	NISSAN	LEAF	Battery Elect	Clean Altern	73	0
5	1G1FW6S08H	Skagit	Concrete	WA	98237	2017	CHEVROLET	BOLT EV	Battery Elect	Clean Altern	238	0
6	3FA6P0SU1K	Snohomish	Everett	WA	98201	2019	FORD	FUSION	Plug-in Hybr	Not eligible	26	0
7	5YJ3E1EB5J	Snohomish	Bothell	WA	98021	2018	TESLA	MODEL 3	Battery Elect	Clean Altern	215	0
8	1N4AZ0CP4D	Snohomish	Everett	WA	98203	2013	NISSAN	LEAF	Battery Elect	Clean Altern	75	0
9	1N4AZ0CP0D	Snohomish	Mukilteo	WA	98275	2013	NISSAN	LEAF	Battery Elect	Clean Altern	75	0
10	1N4BZ0CP4G	Island	Clinton	WA	98236	2016	NISSAN	LEAF	Battery Elect	Clean Altern	84	0
11	KNDJP3AE2G	Skagit	Anacortes	WA	98221	2016	KIA	SOUL	Battery Elect	Clean Altern	93	31950
12	WA1F2AFY1L	Thurston	Lacey	WA	98516	2020	AUDI	Q5 E	Plug-in Hybr	Not eligible	20	0
13	5YJXCBE23L	Grant	Moses Lake	WA	98837	2020	TESLA	MODEL X	Battery Elect	Clean Altern	293	0
14	3C3CFFGE3G	St. Clair	Mascoutah	IL	62258	2016	FIAT	500	Battery Elect	Clean Altern	84	0

Pero te propongo generar un archivo como el anterior, pero que use el punto y coma como separador, en vez de la coma:



```
VIN (1-10);County;City;State;Postal Code;Model Year;Make;Model;Electric Vehicle Typ
JTMEB3FV6N;Monroe;Key West;FL;33040;2022;TOYOTA;RAV4 PRIME;Plug-in Hybrid Electric
1G1RD6E45D;Clark;Laughlin;NV;89029;2013;CHEVROLET;VOLT;Plug-in Hybrid Electric Vehi
JN1AZ0CP8B;Yakima;Yakima;WA;98901;2011;NISSAN;LEAF;Battery Electric Vehicle (BEV);C
1G1FW6S08H;Skagit;Concrete;WA;98237;2017;CHEVROLET;BOLT EV;Battery Electric Vehicle
3FA6P0SU1K;Snhomish;Everett;WA;98201;2019;FORD;FUSION;Plug-in Hybrid Electric Vehi
5YJ3E1EB5J;Snhomish;Bothell;WA;98021;2018;TESLA;MODEL 3;Battery Electric Vehicle (
1N4AZ0CP4D;Snhomish;Everett;WA;98203;2013;NISSAN;LEAF;Battery Electric Vehicle (BE
1N4AZ0CP0D;Snhomish;Mukilteo;WA;98275;2013;NISSAN;LEAF;Battery Electric Vehicle (B
1N4BZ0CP4G;Island;Clinton;WA;98236;2016;NISSAN;LEAF;Battery Electric Vehicle (BEV);
KNDJP3AE2G;Skagit;Anacortes;WA;98221;2016;KIA;SOUL;Battery Electric Vehicle (BEV);C
WA1F2AFY1L;Thurston;Lacey;WA;98516;2020;AUDI;Q5 E;Plug-in Hybrid Electric Vehicle (
```

Para ello, debes diseñar una función que opera con un archivo como el de partida, que usa la coma como separador, y genera otro, con el punto y coma como separador.

```
In [2]: 1  # Esta celda debe ser completada por el estudiante.
        2
        3  def to_semicolon(DatosComas, DatosPunComas):
        4      """
        5          DatosComas : Archivo de origen
        6          DatosPunComas: Nombre del archivo destino
        7
        8          Esta funcion, lee in archivo de origen en formato csv, es decir cuyo separador es una coma ,
        9          y posteriormente escribe ese archivo usando como separador un punto y coma ; en el archivo destino.
       10          El archivo DatosComas debe estar almacenado en la misma carpeta que el Notebook.
       11
       12      """
       13
       14      docnew = ''
       15      f = open(DatosComas, 'r')
       16      for linea in f:
       17          linea = linea.replace(',', ';')
       18          docnew = docnew+linea
       19      f.close()
       20
       21      f2 = open(DatosPunComas, 'w')
       22      f2.write(docnew)
       23      f2.close()
       24
```

```
In [3]: 1  # Ejecución de la función anterior:
        2
        3  DatosComas = "Electric_Vehicle_Population_Data.csv"
        4  DatosPunComas = "Electric_Vehicle_Population_Data_pc.csv"
        5  to_semicolon(DatosComas, DatosPunComas)
```

In [4]:

```
1  # Comprobamos que funciona como es debido, viendo las primeras cinco filas de ambos archivos:
2
3  with open(DatosComas, "r") as f:
4      for _ in range(5):
5          linea = f.readline()
6          print(linea)
7
8  print(".....")
9
10 with open(DatosPunComas, "r") as f:
11     for _ in range(5):
12         linea = f.readline()
13         print(linea)
```

VIN (1-10),County,City,State,Postal Code,Model Year,Make,Model,Electric Vehicle Type,Clean Alternative Fuel Vehicle (CAFV) Eligibility,Electric Range,Base MSRP,Legislative District,DOL Vehicle ID,Vehicle Location,Electric Utility,2020 Census Tract

JTMEB3FV6N,Monroe,Key West,FL,33040,2022,TOYOTA,RAV4 PRIME,Plug-in Hybrid Electric Vehicle (PHEV),Clean Alternative Fuel Vehicle Eligible,42,0,,198968248,POINT (-81.80023 24.5545),,12087972100

1G1RD6E45D,Clark,Laughlin,NV,89029,2013,CHEVROLET,VOLT,Plug-in Hybrid Electric Vehicle (PHEV),Clean Alternative Fuel Vehicle Eligible,38,0,,5204412,POINT (-114.57245 35.16815),,32003005702

JN1AZ0CP8B,Yakima,Yakima,WA,98901,2011,NISSAN,LEAF,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eligible,73,0,15,218972519,POINT (-120.50721 46.60448),PACIFICORP,53077001602

1G1FW6S08H,Skagit,Concrete,WA,98237,2017,CHEVROLET,BOLT EV,Battery Electric Vehicle (BEV),Clean Alternative Fuel Vehicle Eligible,238,0,39,186750406,POINT (-121.7515 48.53892),PUGET SOUND ENERGY INC,53057951101

.....

VIN (1-10);County;City;State;Postal Code;Model Year;Make;Model;Electric Vehicle Type;Clean Alternative Fuel Vehicle (CAFV) Eligibility;Electric Range;Base MSRP;Legislative District;DOL Vehicle ID;Vehicle Location;Electric Utility;2020 Census Tract

JTMEB3FV6N;Monroe;Key West;FL;33040;2022;TOYOTA;RAV4 PRIME;Plug-in Hybrid Electric Vehicle (PHEV);Clean Alternative Fuel Vehicle Eligible;42;0;;198968248;POINT (-81.80023 24.5545);;12087972100

1G1RD6E45D;Clark;Laughlin;NV;89029;2013;CHEVROLET;VOLT;Plug-in Hybrid Electric Vehicle (PHEV);Clean Alternative Fuel Vehicle Eligible;38;0;;5204412;POINT (-114.57245 35.16815);;32003005702

JN1AZ0CP8B;Yakima;Yakima;WA;98901;2011;NISSAN;LEAF;Battery Electric Vehicle (BEV);Clean Alternative Fuel Vehicle Eligible;73;0;15;218972519;POINT (-120.50721 46.60448);PACIFICORP;53077001602

1G1FW6S08H;Skagit;Concrete;WA;98237;2017;CHEVROLET;BOLT EV;Battery Electric Vehicle (BEV);Clean Alternative Fuel Vehicle Eligible;238;0;39;186750406;POINT (-121.7515 48.53892);PUGET SOUND ENERGY INC;53057951101

Nota. En la comprobación anterior, por cada línea que se imprime con la instrucción `print`, se realizan dos saltos de línea. Eso es porque las líneas anteriores se han cargado con la marca `\n`, como puedes ver a continuación, con la última línea. En las funciones que siguen deberás tener esto en cuenta para suprimir la marca `\n` cuando sea necesario.

```
In [5]: 1 linea
```

```
Out[5]: '1G1FW6S08H;Skagit;Concrete;WA;98237;2017;CHEVROLET;BOLT EV;Battery Electric Vehicle (BEV);Clean Alternative
Fuel Vehicle Eligible;238;0;39;186750406;POINT (-121.7515 48.53892);PUGET SOUND ENERGY INC;53057951101\n'
```

a.2) Selección de una línea, separando sus campos

Diseña ahora una función que selecciona una línea y nos da una lista con los valores de sus campos. Los ejemplares de funcionamiento te darán la información sobre cómo deseamos que funcione:

```
In [6]: 1 # Esta celda debe ser completada por el estudiante.
2 def select_line(DatosPunComas, linea):
3     """
4     DatosPunComas: es el nombre del archivo csv o txt, columnas separadas por ;
5     linea: numero de linea que desea ver.
6     Retorna la la linea pedida, con el caracter \n suprimido.
7     """
8     lineNumber = 0
9     listLine = []
10
11     with open(DatosPunComas, "r") as f:
12         for line in f:
13             if (lineNumber == linea):
14                 listLine = (line.strip()).split(';')
15                 #print (listLine)
16                 lineNumber += 1
17         f.close()
18     return listLine
```



```
In [7]: 1 # Comprobación del funcionamiento:
        2
        3 cabecera = select_line(DatosPunComas, 0)
        4 print(cabecera)
        5
        6 linea_1 = select_line(DatosPunComas, 1)
        7 print(linea_1)
```

```
['VIN (1-10)', 'County', 'City', 'State', 'Postal Code', 'Model Year', 'Make', 'Model', 'Electric Vehicle Ty
pe', 'Clean Alternative Fuel Vehicle (CAFV) Eligibility', 'Electric Range', 'Base MSRP', 'Legislative Distri
ct', 'DOL Vehicle ID', 'Vehicle Location', 'Electric Utility', '2020 Census Tract']
['JTMEB3FV6N', 'Monroe', 'Key West', 'FL', '33040', '2022', 'TOYOTA', 'RAV4 PRIME', 'Plug-in Hybrid Electric
Vehicle (PHEV)', 'Clean Alternative Fuel Vehicle Eligible', '42', '0', '', '198968248', 'POINT (-81.80023 2
4.5545)', '', '12087972100']
```

Nota: Observa que se suprime la marca de fin de línea, \n .

a.3) Coordenadas

La cadena 'POINT (-81.80023 24.5545)' indica las coordenadas de un vehículo. Desearíamos convertirla en una función que da, para esa cadena, el par de reales siguiente: (-81.80023 24.5545) .

Pero cuidado: si la cadena fuera una lista vacía u otra cadena con un formato distinto, el valor generado debería ser (-1, -1) .

```
In [9]: 1 # Esta celda debe ser completada por el estudiante.
2 def coords(cadena):
3     """
4     Recibe una cadena con formato: 'POINT (-xx.xxxxx yy.yyyyy)'
5     Retorna un par ordenado '(-xx.xxxxx, yy.yyyyy)'
6     Excepcion: para 'nan' o cadenas vacias retorna '(-1, -1)'
7     """
8     punto = (-1,-1)
9     if (str(cadena) == 'nan') or len(cadena)<1:
10         punto = (-1,-1)
11     else:
12         if (cadena[0:5]=='POINT'):
13             newCad = cadena[7:-1]
14             cadSplited = newCad.split(' ')
15             punto = (float(cadSplited[0]),float(cadSplited[1]))
16             #print (punto)
17     return punto
```

```
In [10]: 1 cadena = 'POINT (-81.80023 24.5545)'
2 punto = coords(cadena)
3
4 print(cadena)
5 print(punto)
6
7 cadena = ''
8 punto = coords(cadena)
9 print(cadena)
10 print(punto)
```

```
POINT (-81.80023 24.5545)
(-81.80023, 24.5545)
```

```
(-1, -1)
```

a.4) Algunos campos son numéricos...

... y sin embargo, al leerlos del archivo de datos, aparecen como cadenas de caracteres:

- La autonomía (Electric Range) debería ser un entero.

- El precio mínimo recomendado por el concesionario (`Base MSRP`) debería ser un entero.
- El distrito legislativo del propietario (`Legislative District`) debería ser un entero.
- La posición (`Vehicle Location`) debería ser un par de reales.

Diseña una función que toma como parámetro una lista con los datos, en formato de caracteres, y cambia los mencionados a los tipos de datos numéricos. Ten cuidado con los datos no consignados (missing): en este caso, imputamos el valor -1, para recordar que se carece de esta información.

```
In [11]: 1  # Esta celda debe ser completada por el estudiante.
2
3  def format_data(List):
4      """
5          Note: Ya que las columnas que deben ser numerica son columnas especificas, yo tomare la posicion de la
6          cada columna como informacion preliminar, luego
7          Columnas:
8          10 : Electric Range
9          11 : Base MSRP
10         12 : Legislative District
11         14 : Vehicle Location'
12         """
13         if (List[10]!=''):
14             List[10] = int(List[10])
15         else:
16             List[10] = -1
17
18         if (List[11]!=''):
19             List[11] = int(List[11])
20         else:
21             List[11] = -1
22
23         if (List[12]!=''):
24             List[12] = int(List[12])
25         else:
26             List[12] = -1
27
28         List[14] = coords(List[14])
29         return List
30
31     print(linea_1)
32
33     linea_1_nueva = format_data(linea_1)
34
35     print(linea_1_nueva)
```

```
['JTMEB3FV6N', 'Monroe', 'Key West', 'FL', '33040', '2022', 'TOYOTA', 'RAV4 PRIME', 'Plug-in Hybrid Electric Vehicle (PHEV)', 'Clean Alternative Fuel Vehicle Eligible', '42', '0', '', '198968248', 'POINT (-81.80023 24.5545)', '', '12087972100']
['JTMEB3FV6N', 'Monroe', 'Key West', 'FL', '33040', '2022', 'TOYOTA', 'RAV4 PRIME', 'Plug-in Hybrid Electric Vehicle (PHEV)', 'Clean Alternative Fuel Vehicle Eligible', 42, 0, -1, '198968248', (-81.80023, 24.5545), '', '12087972100']
```

In [12]:

```
1 print(linea_1)
2
3 linea_1_nueva = format_data(linea_1)
4
5 print(linea_1_nueva)
```

```
['JTMEB3FV6N', 'Monroe', 'Key West', 'FL', '33040', '2022', 'TOYOTA', 'RAV4 PRIME', 'Plug-in Hybrid Electric Vehicle (PHEV)', 'Clean Alternative Fuel Vehicle Eligible', 42, 0, -1, '198968248', (-81.80023, 24.5545), '', '12087972100']
['JTMEB3FV6N', 'Monroe', 'Key West', 'FL', '33040', '2022', 'TOYOTA', 'RAV4 PRIME', 'Plug-in Hybrid Electric Vehicle (PHEV)', 'Clean Alternative Fuel Vehicle Eligible', 42, 0, -1, '198968248', (-1, -1), '', '12087972100']
```

b) Dos cálculos básicos [2 puntos]

b.1) Cómputo de vehículos por estado

Dado un estado, diseña una función que calcule cuántos vehículos electricos hay registrados de dicho estado, a partir del archivo `csv` con punto y comas.

```
In [13]: 1  # Esta celda debe ser completada por el estudiante.
2  #
3
4  def num_ve_estado(DatosPunComas, Est):
5      """
6
7      """
8      count = 0
9      lenDatos = len(DatosPunComas)
10
11     with open(DatosPunComas, 'r') as f:
12         for line in f:
13             linea = (line.strip()).split(';')
14             #print (linea[3])
15             if (linea[3]==Est):
16                 count +=1
17
18     return count
19
20 print(num_ve_estado(DatosPunComas, "FL"))
21 print(num_ve_estado(DatosPunComas, "TX"))
```

6
14

```
In [14]: 1  # Comprobación de funcionamiento, con los estados de Florida y Texas:
2
3  # Florida:
4  print(num_ve_estado(DatosPunComas, "FL"))
5
6  # Texas:
7  print(num_ve_estado(DatosPunComas, "TX"))
```

6
14

b.2) Cómputo de vehículos de todos los estados

Para cada estado, cuántos vehículos electricos tiene registrados, a partir del archivo csv con punto y comas.

Cuidado: esta operación no debería hacer más que un recorrido de la tabla por razones de eficiencia.

```
In [15]: 1  # Esta celda debe ser completada por el estudiante.
2  def num_ve_todos_estados(DatosPunComas):
3      """
4
5      """
6      Estados = []
7      NumVeEstados = []
8      lenDatos = len(DatosPunComas)
9
10     with open(DatosPunComas, "r") as f:
11         i=0
12         for line in f:
13             i +=1
14             linea = (line.strip()).split(';')
15             Estados.append(linea[3])
16             NumVeEstados.append(1)
17     #print(Estados,NumVeEstados )
18     print ( ' lines',i)
19     f.close()
20     # Asignando el primer estado como elemento en un diccionario de datos.
21     # usando 1 para evitar la linea 0 que corresponde a los titulos de las columnas.
22     Dic_Estados = {Estados[1]:NumVeEstados[1]}
23
24     for i in range(2,len(Estados)):
25         Dic_tmp = {Estados[i]:NumVeEstados[i]}
26         newkey = Estados[i]
27         if newkey in Dic_Estados:
28             Dic_Estados[newkey] = Dic_Estados[newkey] + 1
29         else:
30             Dic_Estados[newkey] = 1
31
32     print (Dic_Estados)
33
34     #num_ve_todos_estados(DatosPunComas)
```

```
In [16]: 1 # Comprobación de funcionamiento:
        2
        3 print(num_ve_todos_estados(DatosPunComas))

lines 112635
{'FL': 6, 'NV': 8, 'WA': 112348, 'IL': 6, 'NY': 4, 'VA': 36, 'OK': 1, 'KS': 3, 'CA': 76, 'NE': 5, 'MD': 26,
'CO': 9, 'DC': 6, 'TN': 3, 'SC': 5, 'CT': 6, 'OR': 5, 'TX': 14, 'SD': 1, 'HI': 4, 'GA': 7, 'MS': 1, 'AR': 4,
'NC': 7, 'MO': 3, 'UT': 4, 'PA': 3, 'DE': 1, 'OH': 2, 'WY': 2, 'AL': 1, 'ID': 2, 'AZ': 6, 'AK': 1, 'LA': 3,
'NM': 1, 'WI': 1, 'KY': 1, 'NJ': 3, 'MN': 1, 'MA': 3, 'ME': 1, 'RI': 1, 'NH': 2, 'ND': 1}
None
```

b.3) Cálculo de la autonomía media de los vehículos en cada año

Diseña una función que calcule la autonomía media (redondeado a dos decimales) de los vehículos registrados en cada año, a partir del archivo csv con punto y comas.

Las autonomías de vehículos consignadas con el valor cero no deben tenerse en cuenta, al tratarse de datos missing en realidad.

In [22]:

```
1  # Esta celda debe ser completada por el estudiante.
2
3
4  # Funcion para leer la cabecera de la tabla, para encontrar la columna correspondiente a Autonomia.
5
6  def findcolumn(cadena):
7      """
8      Esta funcion encuentra el indice de la columna correspondiente
9      a la cadena dada como parametro de entrada.
10     """
11     cabecera = select_line(DatosPunComas, 0)
12     #print(cabecera)
13     lc = len(cabecera)
14     for i in range(lc):
15         if cabecera[i]== cadena:
16             return i
17     return -1
18
19
20 def aut_medias(DatosPunComas):
21     """
22     DatosPunComas: archivo csv con punto y comas.
23     Esta funcion retorna la autonomía media (redondeado a dos decimales) de los vehículos registrados en c
24     a partir del archivo DatosPunComas.
25     """
26     index1 = findcolumn('Model Year')
27     index2 = findcolumn('Electric Range')
28     Autonomias = dict()
29     Dic_Years = dict()
30
31     with open(DatosPunComas, "r") as f:
32         linea = f.readline()
33         #print(linea)
34         i=0
35         for line in f:
36             i +=1
37             elementos = (line.strip()).split(';')
38             Autonomias [elementos[0]+str(i)] = { 'Year': elementos[index1],
39                                                 'Electric Range': float(elementos[index2])}
40     f.close()
41     #print (len(Autonomias))
```

```

42     #print ( ' ',i)
43
44     for i, er in Autonomias.items():
45         #print (er)
46         y = int(er['Year'])
47         erval = er['Electric Range']
48         if (erval>0):
49             Dic_Years[y] = [erval] if y not in Dic_Years.keys() else Dic_Years[y] + [erval]
50
51     for y, er in Dic_Years.items():
52         mean_er = np.mean(er)
53         Dic_Years [y] = np.round((mean_er),2)
54
55     #print(Dic_Years)
56     return Dic_Years
57
58 #autonom_medias = aut_medias(DatosPunComas)
59 #print(autonom_medias)

```

```

In [23]: 1  # Comprobación de funcionamiento:
          2
          3 autonom_medias = aut_medias(DatosPunComas)
          4
          5 print(autonom_medias)

```

```
{2022: 28.86, 2013: 81.04, 2011: 71.24, 2017: 111.56, 2019: 176.99, 2018: 155.73, 2016: 101.24, 2020: 243.2
5, 2021: 48.25, 2015: 97.87, 2014: 81.71, 2012: 61.73, 2010: 245.0, 2023: 31.65, 2008: 220.0, 2000: 58.0, 19
97: 39.0, 1999: 74.0, 1998: 58.0, 2002: 95.0}
```

b.4) Lo mismo, en orden ascendente

Pasa los resultados anteriores a una lista en orden ascendente.

```

In [27]: 1 # Esta celda debe ser completada por el estudiante.
          2
          3 def ArrayParesOrdenados(DicPares):
          4     """
          5     Esta funcion construye una lista a partir de un diccionario de pares ordenados.
          6
          7     """
          8     #Ordenando el diccionario.
          9     DicPares1 = dict(sorted(DicPares.items()))
         10     ListDicPares = []
         11
         12     for i, er in DicPares1.items():
         13         ListDicPares.append((i,er))
         14     return ListDicPares
         15
         16 lista_aut_medias = ArrayParesOrdenados(autonom_medias)
         17

```

```

In [28]: 1 # Comprobación de funcionamiento:
          2
          3 print(lista_aut_medias)

```

```

[(1997, 39.0), (1998, 58.0), (1999, 74.0), (2000, 58.0), (2002, 95.0), (2008, 220.0), (2010, 245.0), (2011,
71.24), (2012, 61.73), (2013, 81.04), (2014, 81.71), (2015, 97.87), (2016, 101.24), (2017, 111.56), (2018, 1
55.73), (2019, 176.99), (2020, 243.25), (2021, 48.25), (2022, 28.86), (2023, 31.65)]

```

c) Algunas gráficas [1 punto]

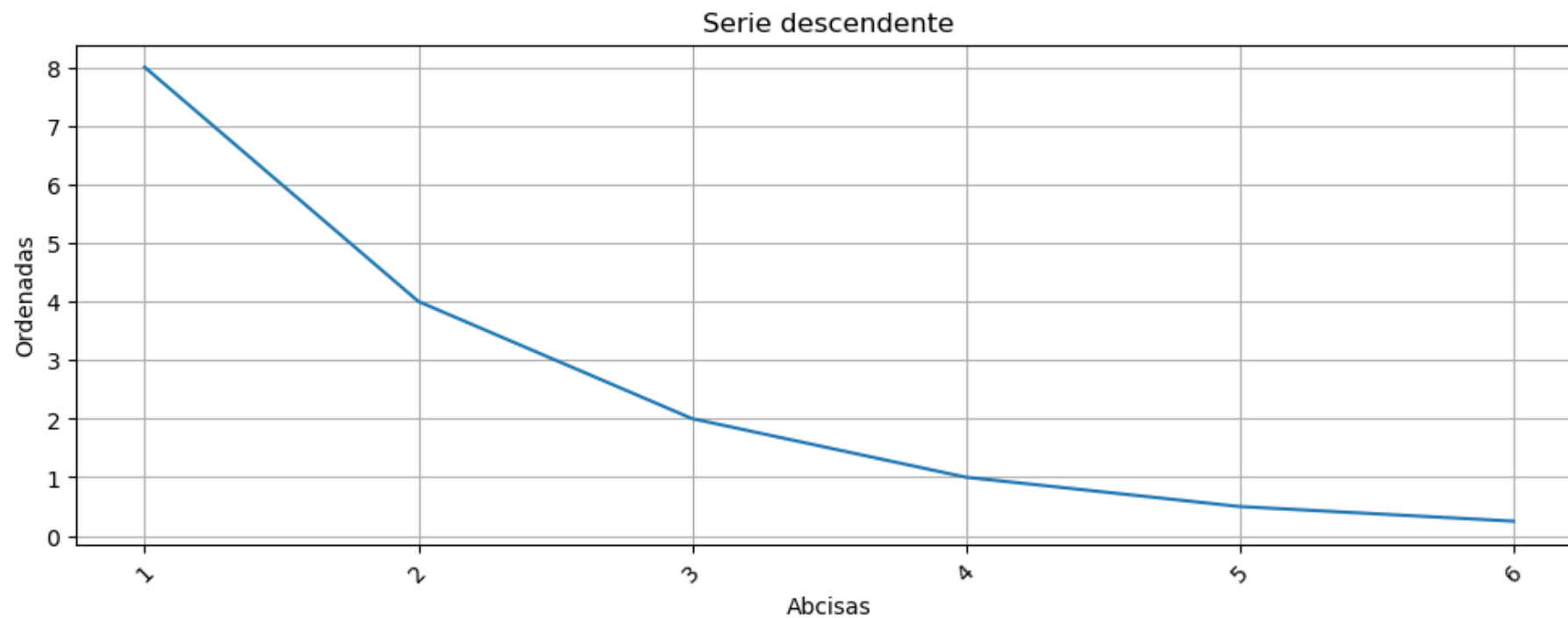
c.1 Un modelo de gráfica

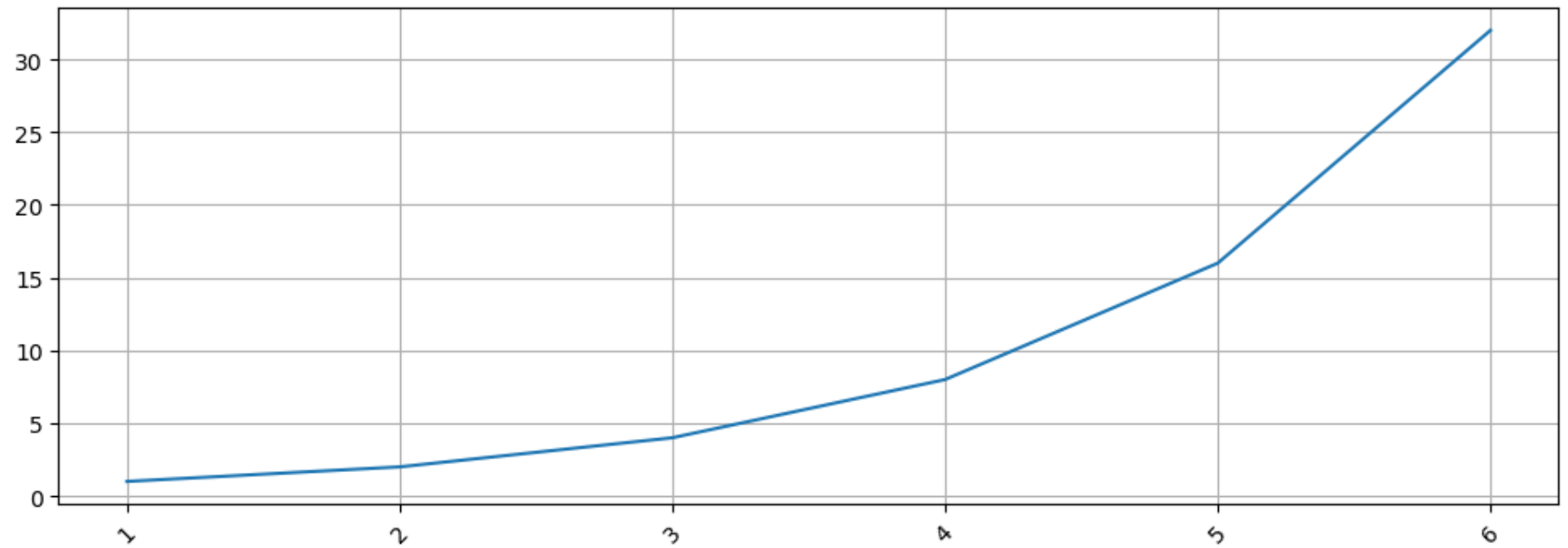
Vamos a diseñar un modelo de gráfica sencillo que nos sirva para las siguientes representaciones. Tomará como parámetro una lista de pares (x, y) , y opcionalmente los tres rótulos explicativos que necesitamos incluir. Además, queremos que las etiquetas de las abcisas aparezcan inclinadas, para poder luego mostrar intervalos de edad.

Las pruebas de funcionamiento te darán más información que las explicaciones que pueda yo dar aquí.

```
In [29]: 1  # Esta celda debe ser completada por el estudiante.
2
3  def representar_xxx_yyy(arrayP, titulo = ['', '', '']): # titulos opcionales
4      """
5      Recibe:
6          arrayP: Arreglo de pares ordenados a graficar.
7          titulo: Cadena titulo del grafico. [Opcional]
8
9      Funcion crea un grafico a partir de arrayP.
10     Muestra grafico.
11
12     """
13     a = np.array(arrayP)
14     x = np.transpose(a)[0]
15     y = np.transpose(a)[1]
16     plt.figure(figsize=(12,4))
17     plt.plot( y, '-')
18     plt.title(titulo[0])
19     plt.ylabel(titulo[1])
20     plt.xlabel(titulo[2])
21     plt.grid(True)
22     plt.xscale('linear')
23     x_pos = np.arange(len(x))
24     x_labels = [int(i) for i in x]
25     plt.xticks(x_pos, x_labels, rotation=45)
26     plt.show()
27
```

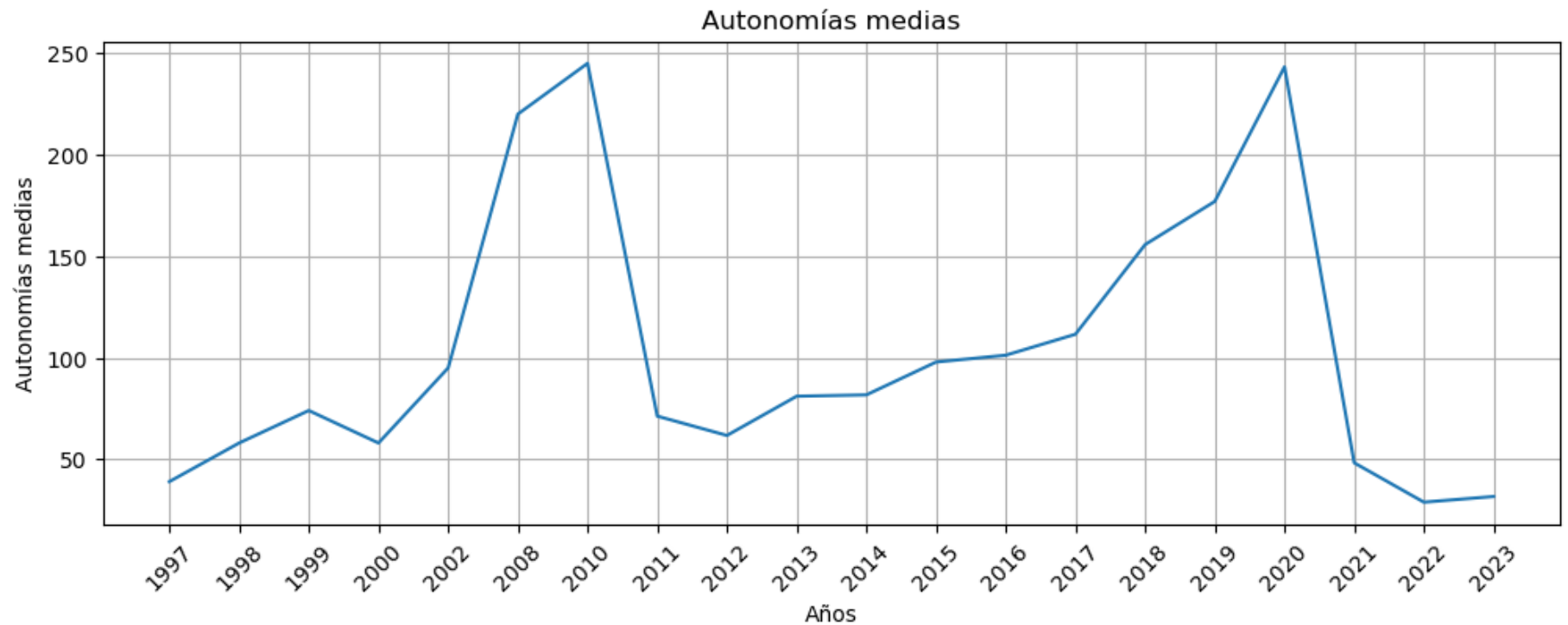
```
In [30]: 1 # Pruebas de funcionamiento:
2
3 representar_xxx_yyy([(1, 8), (2, 4), (3, 2), (4, 1), (5, 0.5), (6, 0.25)], ["Serie descendente", "Ordenada
4
5 representar_xxx_yyy([(1, 1), (2, 2), (3, 4), (4, 8), (5, 16), (6, 32)])
```





Lógicamente, hemos diseñado nuestro modelo para aplicarlo posteriormente a los datos que ya tenemos. Concretamente, a la representación de las autonomías medias de los autotmóviles registrados en cada año.

```
In [32]: 1 # Pruebas de funcionamiento:
2
3 representar_xxx_yyy(lista_aut_medias, ["Autonomías medias", "Autonomías medias", "Años"])
```



c.2 Replanteamiento de la gráfica anterior

Vemos ahora que la curva se comporta de un modo extraño. Debería ser creciente, pero no es así. Quizá estamos comparando tecnologías y marcas diferentes.

Para ver si esta puede ser la razón, hagamos el cálculo y la gráfica para vehículos Nissan, y únicamente con tecnología "Battery Electric Vehicle (BEV)".

In [35]:

```
1  # Esta celda debe ser completada por el estudiante.
2
3  def aut_medias_conFiltro(DatosPunComas, Maker, tecnologia):
4      """
5      Esta funcion crea un grafico a partir del archivo 'DatosPunComas',
6      usando la columna 'Maker' y la columna 'tecnologia'.
7      """
8
9      index1 = findcolumn('Model Year')
10     index2 = findcolumn('Electric Range')
11     indexF1 = findcolumn('Make')
12     indexF2 = findcolumn('Electric Vehicle Type')
13
14     f = open(DatosPunComas, "r")
15     linea = f.readline()
16     #print(linea)
17     Autonomias = dict()
18     for linea in f:
19         elementos = (linea.strip()).split(';')
20         # Aplicar un filtro por Casa Automotriz y tecnologia:
21         if (elementos[indexF1] == Maker) and (elementos[indexF2] == tecnologia):
22             Autonomias [elementos[0]] = { 'Year': elementos[index1], 'Electric Range': float(elementos[inc
23     f.close()
24     #print (Autonomias)
25
26
27     Dic_Years = dict()
28     for i, er in Autonomias.items():
29         #print (er)
30         y = int(er['Year'])
31         erval = er['Electric Range']
32         if (erval>0):
33             Dic_Years[y] = [erval] if y not in Dic_Years.keys() else Dic_Years[y] + [erval]
34
35
36     for y, er in Dic_Years.items():
37         mean_er = np.mean(er)
38         Dic_Years [y] = np.round((mean_er),2)
39
40
41     #print(Dic_Years)
```

```

42     return Dic_Years
43
44 aut_medias_Nissan_BEV = aut_medias_conFiltro(DatosPunComas, 'NISSAN', "Battery Electric Vehicle (BEV)")
45 print(aut_medias_Nissan_BEV)
46
47 lista_aut_medias_Nissan_BEV = ArrayParesOrdenados(aut_medias_Nissan_BEV)
48

```

```

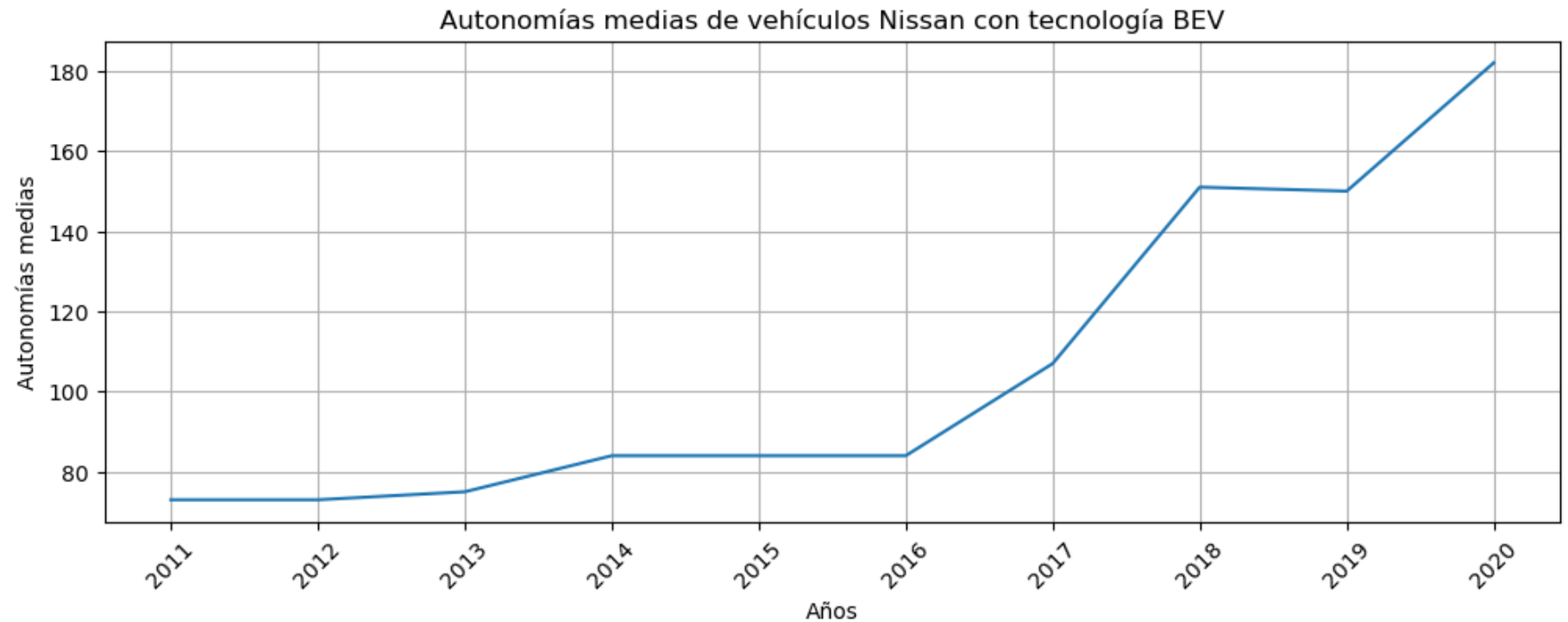
{2011: 73.0, 2013: 75.0, 2016: 84.0, 2019: 150.0, 2017: 107.0, 2018: 151.0, 2015: 84.0, 2012: 73.0, 2014: 84.0, 2020: 182.0}

```

```

In [34]: 1 # Pruebas de funcionamiento:
2
3 representar_xxx_yyy(lista_aut_medias_Nissan_BEV, \
4                  ["Autonomías medias de vehículos Nissan con tecnología BEV", "Autonomías medias", "Años

```



d) Operaciones con dataframes [3 puntos]

En este apartado, vamos a trabajar con tablas de la librería `pandas`, llamadas `dataframes`.

d.1) Carga del dataframe

La primera operación que necesitamos es cargar el archivo de datos en una tabla, como se ve en el siguiente ejemplo.

```
In [36]: 1 # Esta celda debe ser completada por el estudiante.
          2 def cargar_dataframe(DatosPunComas):
          3     """
          4     Esta funcion lee el archivo 'DatosPunComas' y lo asigna a un
          5     pandas dataframe.
          6     Retorna el dataframe.
          7     """
          8     data = pd.read_csv(DatosPunComas, sep = ';')
          9
         10     return data
         11
```

In [37]:

```
1 # Comprobación
2
3 tabla_pre = cargar_dataframe(DatosPunComas)
4
5 tabla_pre
```

112630	1N4BZ1CP7K	San Juan	Friday Harbor	WA	98250	2019	NISSAN	LEAF	Battery Electric Vehicle (BEV)	Clean Alternative Fuel Vehicle Eligible	150	0	40.0	10366
112631	1FMCU0KZ4N	King	Vashon	WA	98070	2022	FORD	ESCAPE	Plug-in Hybrid Electric Vehicle (PHEV)	Clean Alternative Fuel Vehicle Eligible	38	0	34.0	19387
112632	KNDCD3LD4J	King	Covington	WA	98042	2018	KIA	NIRO	Plug-in Hybrid Electric Vehicle (PHEV)	Not eligible due to low battery range	26	0	47.0	12503
112633	YV4BR0CL8N	King	Covington	WA	98042	2022	VOLVO	XC90	Plug-in Hybrid Electric Vehicle (PHEV)	Not eligible due to low battery range	18	0	47.0	19467

d.2) Simplificación de la tabla

Ahora queremos descartar los datos de vehículos de Washington, y también queremos quedarnos únicamente con con las columnas del estado, año del modelo, marca, autonomía y localización.

```

In [38]: 1 # Esta celda debe ser completada por el estudiante.
          2
          3 tabla_pre.columns
          4
          5 tabla2 = tabla_pre[['State', 'Model Year', 'Make', 'Electric Range', 'Vehicle Location' ]]
          6 tabla2
          7
          8 tabla = tabla2.loc[tabla2['State'] != 'WA']
          9 tabla

```

Out[38]:

	State	Model Year	Make	Electric Range	Vehicle Location
0	FL	2022	TOYOTA	42	POINT (-81.80023 24.5545)
1	NV	2013	CHEVROLET	38	POINT (-114.57245 35.16815)
12	IL	2016	FIAT	84	POINT (-89.79939 38.49028)
19	NY	2016	TESLA	200	POINT (-73.84643 43.1284)
41	VA	2016	TESLA	210	POINT (-76.53585 37.10499)
...	...	...	...	...	...
112153	NH	2015	CHEVROLET	38	POINT (-71.12513 44.04945)
112301	SC	2021	TESLA	0	POINT (-80.17601 33.01897)
112394	KS	2017	BMW	14	POINT (-94.89874 39.23762)
112541	ND	2018	MITSUBISHI	22	POINT (7.86484 51.32975)
112603	ID	2022	RIVIAN	0	POINT (-116.91895 47.40077)

286 rows × 5 columns

```
In [39]: 1 # Comprobación
        2 tabla
```

Out[39]:

	State	Model Year	Make	Electric Range	Vehicle Location
0	FL	2022	TOYOTA	42	POINT (-81.80023 24.5545)
1	NV	2013	CHEVROLET	38	POINT (-114.57245 35.16815)
12	IL	2016	FIAT	84	POINT (-89.79939 38.49028)
19	NY	2016	TESLA	200	POINT (-73.84643 43.1284)
41	VA	2016	TESLA	210	POINT (-76.53585 37.10499)
...	...	...	...	...	...
112153	NH	2015	CHEVROLET	38	POINT (-71.12513 44.04945)
112301	SC	2021	TESLA	0	POINT (-80.17601 33.01897)
112394	KS	2017	BMW	14	POINT (-94.89874 39.23762)
112541	ND	2018	MITSUBISHI	22	POINT (7.86484 51.32975)
112603	ID	2022	RIVIAN	0	POINT (-116.91895 47.40077)

286 rows × 5 columns

d.3) Comprobación de tipos y otras cosillas

Comprueba que el año y la autonomía son enteros, imprimiendo el valor y tipo de estos datos en la primera fila:

```
In [40]: 1 # Esta celda debe ser completada por el estudiante.
        2 #print (tabla.columns)
        3
        4 print ((tabla['Model Year']).iloc[0], (tabla['Model Year'].iloc[0]).dtype)
        5 print ((tabla['Electric Range']).iloc[0], (tabla['Electric Range'].iloc[0]).dtype)
```

```
2022 int64
42 int64
```

Convierte ahora las coordenadas de la columna entera, según describimos antes:


```
In [41]: 1 # Esta celda debe ser completada por el estudiante.
2
3 Puntos_df = pd.DataFrame(tabla['Vehicle Location'], columns = ['Vehicle Location'])
4 for i in range(len(tabla)):
5     cadena = str(tabla['Vehicle Location'].iloc[i])
6     punto_i = coords(cadena)
7     Puntos_df['Vehicle Location'].iloc[i] = punto_i
8
9
10 tabla['Vehicle Location'] = Puntos_df['Vehicle Location']
11 tabla
```

/var/folders/xh/_42kc4k13kb5l4wd0dzs80r00000gp/T/ipykernel_18164/1074898773.py:10: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
tabla['Vehicle Location'] = Puntos_df['Vehicle Location']
```

Out[41]:

	State	Model Year	Make	Electric Range	Vehicle Location
0	FL	2022	TOYOTA	42	(-81.80023, 24.5545)
1	NV	2013	CHEVROLET	38	(-114.57245, 35.16815)
12	IL	2016	FIAT	84	(-89.79939, 38.49028)
19	NY	2016	TESLA	200	(-73.84643, 43.1284)
41	VA	2016	TESLA	210	(-76.53585, 37.10499)
...	...	...	...	...	...
112153	NH	2015	CHEVROLET	38	(-71.12513, 44.04945)
112301	SC	2021	TESLA	0	(-80.17601, 33.01897)
112394	KS	2017	BMW	14	(-94.89874, 39.23762)
112541	ND	2018	MITSUBISHI	22	(7.86484, 51.32975)
112603	ID	2022	RIVIAN	0	(-116.91895, 47.40077)

286 rows × 5 columns

In [42]:

```
1 # Comprobación:
2 tabla
```

Out[42]:

	State	Model Year	Make	Electric Range	Vehicle Location
0	FL	2022	TOYOTA	42	(-81.80023, 24.5545)
1	NV	2013	CHEVROLET	38	(-114.57245, 35.16815)
12	IL	2016	FIAT	84	(-89.79939, 38.49028)
19	NY	2016	TESLA	200	(-73.84643, 43.1284)
41	VA	2016	TESLA	210	(-76.53585, 37.10499)
...	...	...	...	...	...
112153	NH	2015	CHEVROLET	38	(-71.12513, 44.04945)
112301	SC	2021	TESLA	0	(-80.17601, 33.01897)
112394	KS	2017	BMW	14	(-94.89874, 39.23762)
112541	ND	2018	MITSUBISHI	22	(7.86484, 51.32975)
112603	ID	2022	RIVIAN	0	(-116.91895, 47.40077)

286 rows × 5 columns

Ahora deseamos contabilizar cuántos registros tenemos de cada marca, formando una tabla como la que se muestra más abajo.

```
In [43]: 1 # Esta celda debe ser completada por el estudiante.
2 lt = len(tabla)
3 print (lt)
4 tabla_freqs = pd.DataFrame({'Marca':tabla['Make'].values , 'Freq':np.ones(lt)})
5 tabla_freqs = tabla_freqs.groupby(['Marca'], as_index = False).count()
6 #tabla_freqs
7
8 tabla_freqs = tabla_freqs.sort_values(by =['Freq'], ascending = False)
9 tabla_freqs = tabla_freqs.reset_index(drop=True)
10
```

286

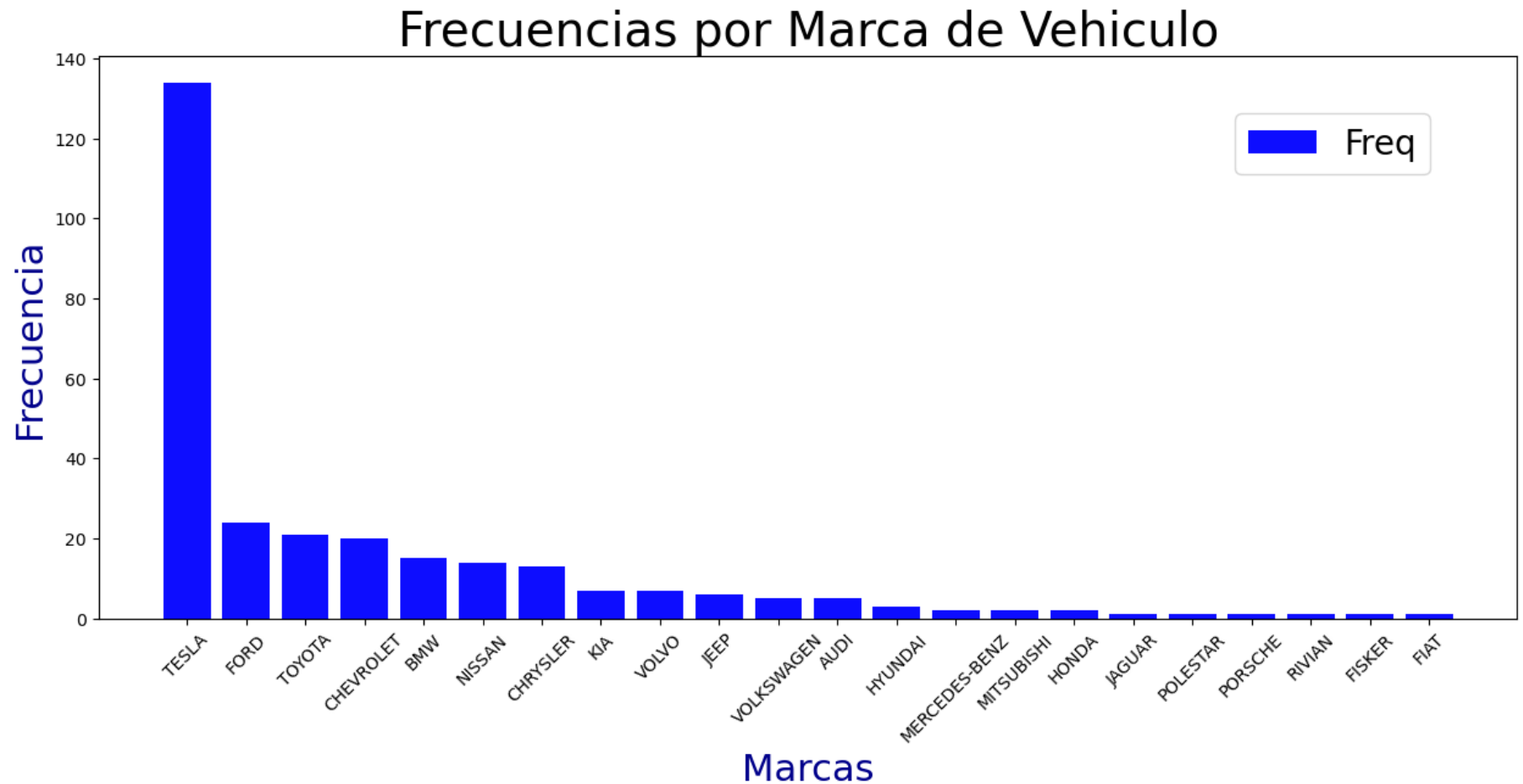
```
In [44]: 1 # Comprobación:
2
3 tabla_freqs
```

Out[44]:

	Marca	Freq
0	TESLA	134
1	FORD	24
2	TOYOTA	21
3	CHEVROLET	20
4	BMW	15
5	NISSAN	14
6	CHRYSLER	13
7	KIA	7
8	VOLVO	7
9	JEEP	6
10	VOLKSWAGEN	5

Y ya sólo nos queda mostrar el resultado obtenido en un gráfico apropiado:

```
In [45]: 1  # Esta celda debe ser completada por el estudiante.
2
3
4  plt.figure(figsize=(15,6))
5
6  plt.title('Frecuencias por Marca de Vehiculo', { 'fontsize':28, 'color':'black'})
7  plt.ylabel('Frecuencia', { 'fontsize':22, 'color':'darkblue'})
8  plt.xlabel('Marcas',{ 'fontsize':22, 'color':'darkblue'})
9  plt.grid(False)
10 plt.xscale('linear')
11 x_pos = np.arange(len(tabla_fregs.Marca,))
12 x_labels = [str(i) for i in tabla_fregs.Marca]
13 plt.bar( x_pos, tabla_fregs.Freq, align='center', color='b',alpha=0.95, label='Freq')
14 plt.xticks( x_pos, x_labels, rotation=45)
15 plt.legend( bbox_to_anchor=(0.80, 0.90), loc='upper left',
16             borderaxespad=0.05, fontsize= 20)
17
18 plt.show()
19
```



e) Un cálculo masivo con map-reduce [0.5 puntos]

En este apartado se ha de realizar un programa aparte que calcule, para cada estado, el número de vehículos eléctricos registrados, junto con la autonomía máxima y su marca:

```
C:\...> python veh_el_autonom.py -q Electric_Vehicle_Population_Data_pc.csv
```

El programa funcionará necesariamente con la técnica map-reduce, que podemos poner en juego con la librería `mrjob`.

El funcionamiento del mismo se puede activar también desde aquí:

```
1 pip install --trusted-host pypi.org --trusted-host files.pythonhosted.org mrjob
```

```
In [46]: 1 # Hagamos una llamada al programa de consola desde aquí:
          2 ! python "veh_el_autonom.py" "Data_pc_noheader.txt" -q
```


"WI"	[1, "TESLA", "0"]
"WY"	[2, "TESLA", "322"]
"AK"	[1, "FORD", "19"]
"AL"	[1, "TOYOTA", "42"]
"AR"	[4, "TESLA", "322"]
"AZ"	[6, "TESLA", "208"]
"CA"	[76, "TESLA", "322"]
"CO"	[9, "TESLA", "330"]
"CT"	[6, "TOYOTA", "42"]
"DC"	[6, "TESLA", "322"]
"DE"	[1, "TESLA", "0"]
"FL"	[6, "TESLA", "220"]
"GA"	[7, "TESLA", "215"]
"HI"	[4, "TESLA", "270"]
"ID"	[2, "RIVIAN", "0"]
"IL"	[6, "TESLA", "322"]
"KS"	[3, "BMW", "14"]
"KY"	[1, "CHRYSLER", "33"]
"LA"	[3, "BMW", "72"]
"MA"	[3, "CHEVROLET", "53"]
"MD"	[26, "TESLA", "291"]
"ME"	[1, "CHRYSLER", "33"]
"MN"	[1, "TESLA", "220"]
"MO"	[3, "TESLA", "322"]
"MS"	[1, "TESLA", "0"]
"NC"	[7, "CHEVROLET", "53"]
"ND"	[1, "MITSUBISHI", "22"]
"NE"	[5, "TESLA", "322"]
"NH"	[2, "TESLA", "308"]
"NJ"	[3, "HYUNDAI", "29"]
"NM"	[1, "CHEVROLET", "238"]
"NV"	[8, "TESLA", "291"]
"NY"	[4, "TESLA", "200"]
"OH"	[2, "NISSAN", "0"]
"OK"	[1, "BMW", "72"]
"OR"	[5, "CHEVROLET", "238"]
"PA"	[3, "TESLA", "200"]
"RI"	[1, "NISSAN", "84"]
"SC"	[5, "TOYOTA", "25"]
"SD"	[1, "TESLA", "0"]
"TN"	[3, "TESLA", "308"]

```
"TX"      [14, "TESLA", "322"]
"UT"      [4, "TESLA", "215"]
"VA"      [36, "TESLA", "330"]
"WA"      [112348, "TESLA", "337"]
```

```
In [47]: 1 # Para que el resultado se almacene en un archivo:
          2
          3 ! python "veh_el_autonom.py" "Data_pc_noheader.txt" -q > datos_vehículos.txt
```

```
In [48]: 1 # Veamos el archivo generado:
          2 ! cat datos_vehículos.txt
```

"WI"	[1, "TESLA", "0"]
"WY"	[2, "TESLA", "322"]
"AK"	[1, "FORD", "19"]
"AL"	[1, "TOYOTA", "42"]
"AR"	[4, "TESLA", "322"]
"AZ"	[6, "TESLA", "208"]
"CA"	[76, "TESLA", "322"]
"CO"	[9, "TESLA", "330"]
"CT"	[6, "TOYOTA", "42"]
"DC"	[6, "TESLA", "322"]
"DE"	[1, "TESLA", "0"]
"FL"	[6, "TESLA", "220"]
"GA"	[7, "TESLA", "215"]
"HI"	[4, "TESLA", "270"]
"ID"	[2, "RIVIAN", "0"]
"IL"	[6, "TESLA", "322"]
"KS"	[3, "BMW", "14"]
"KY"	[1, "CHRYSLER", "33"]
"LA"	[3, "BMW", "72"]
"MA"	[3, "CHEVROLET", "53"]
"MD"	[26, "TESLA", "291"]
"ME"	[1, "CHRYSLER", "33"]
"MN"	[1, "TESLA", "220"]
"MO"	[3, "TESLA", "322"]
"MS"	[1, "TESLA", "0"]
"NC"	[7, "CHEVROLET", "53"]
"ND"	[1, "MITSUBISHI", "22"]
"NE"	[5, "TESLA", "322"]
"NH"	[2, "TESLA", "308"]
"NJ"	[3, "HYUNDAI", "29"]
"NM"	[1, "CHEVROLET", "238"]
"NV"	[8, "TESLA", "291"]
"NY"	[4, "TESLA", "200"]
"OH"	[2, "NISSAN", "0"]
"OK"	[1, "BMW", "72"]
"OR"	[5, "CHEVROLET", "238"]
"PA"	[3, "TESLA", "200"]
"RI"	[1, "NISSAN", "84"]
"SC"	[5, "TOYOTA", "25"]
"SD"	[1, "TESLA", "0"]
"TN"	[3, "TESLA", "308"]

```
"TX"      [14, "TESLA", "322"]
"UT"      [4, "TESLA", "215"]
"VA"      [36, "TESLA", "330"]
"WA"      [112348, "TESLA", "337"]
```

Para que pueda yo ver tu programa cómodamente desde aquí, también se puede mostrar con un comando de la consola, anteponiendo el símbolo `!`. Observaciones:

- La instrucción siguiente está comentada para ocultar una solución mía. Tú debes suprimir el símbolo `#` del comentario para mostrar tu solución aquí.
- Desde mac o linux, se ha de usar el comando `cat`, en vez de `type`.

In [49]:

```
1 ! cat veh_el_autonom.py
2
3
```

```
#programa aparte que calcule, para cada estado, el número de vehículos eléctricos registrados, junto con la
a autonomía máxima y su marca:
#El programa funcionará necesariamente con la técnica map-reduce, que podemos poner en juego con la librería
mrjob.
```

```
import sys
from mrjob.job import MRJob
```

```
def calc_max(values):
    """
    example [1, 'TSLA', 5], [1, 'RIVIAN', 4], [1, 'TSLA', 3] ----> [3, 'TSLA', 5]
    """
    vmax = 0
    vcount = 0
    make = ""
    for i, j, k in values:
        if (float(k) >= float(vmax)):
            vmax = k
            make = j
    return make, vmax
```

f) Un apartado libre [0.5 puntos]

Usar la libreria de geopandas, para mostrar un mapa de norte america con todos los vehiculos vendidos, por marca (Maker).

```
1 pip install --trusted-host pypi.org --trusted-host files.pythonhosted.org geopandas
```

```
In [50]: 1 tabla.columns
```

```
Out[50]: Index(['State', 'Model Year', 'Make', 'Electric Range', 'Vehicle Location'], dtype='object')
```

In [117]:

```
1 def sepLatLong(column_coors):
2     """
3     Esta funcion recibe la columna con las coordenadas geograficas en
4     formato (-+x.xxx, -+y.yyy), y las separa en dos columnas separdas,
5     para latitud y longitud.
6     Retorna dos arreglos, en el orden de latitud y longitud.
7
8     """
9     ccl = len(column_coors)
10    lat = np.zeros(ccl)
11    lon = np.zeros(ccl)
12    i=0
13    for v in (column_coors):
14        lat[i], lon[i] = float(v[0]), float(v[1])
15        #print (lat[i], lon[i])
16        i +=1
17    return lat, lon
18
19 ccl = len(tabla)
20 tabla['Latitude'] = np.zeros(ccl)
21 tabla['Longitude'] = np.zeros(ccl)
22
23 tabla['Latitude'], tabla['Longitude'] = sepLatLong(tabla['Vehicle Location'])
24
```

```
/var/folders/xh/_42kc4k13kb5l4wd0dzs80r00000gp/T/ipykernel_84614/2461843827.py:15: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
    tabla['Latitude'] = np.zeros(ccl)  
/var/folders/xh/_42kc4k13kb5l4wd0dzs80r00000gp/T/ipykernel_84614/2461843827.py:16: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
    tabla['Longitude'] = np.zeros(ccl)  
/var/folders/xh/_42kc4k13kb5l4wd0dzs80r00000gp/T/ipykernel_84614/2461843827.py:18: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

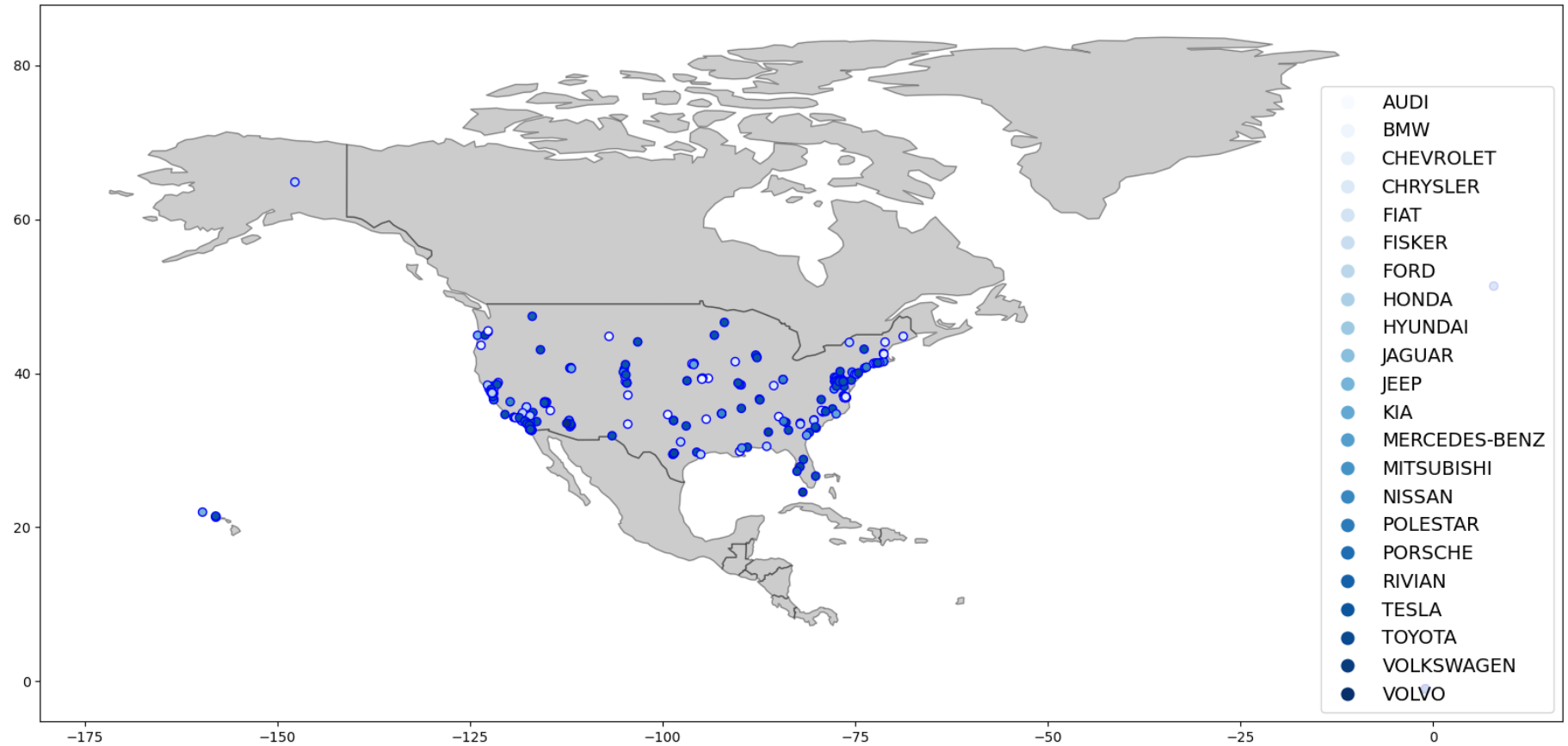
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy (https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
    tabla.loc[:, 'Latitude'], tabla.loc[:, 'Longitude'] = sepLatLong(tabla['Vehicle Location'])
```



```
In [106]: 1
2  gdf = geopandas.GeoDataFrame(
3          tabla, geometry = geopandas.points_from_xy(
4              tabla.Latitude, tabla.Longitude))
5
6  world = geopandas.read_file(geopandas.datasets.get_path('naturalearth_lowres'))
7
```

Ubicacion de las diferentes marcas



```
In [40]: 1 # Pruebas de funcionamiento:
2 fig, ax = plt.subplots(figsize=(20, 20))
3 world[world.continent == 'North America'].plot(ax = ax,
4         figsize=(15,15), alpha = 0.4,
5         color='gray', edgecolor='black')
6
7 gdf.plot(column = "Make", cmap='Blues', edgecolor='blue', ax=ax,
8         legend = True, legend_kwds={'loc':'lower right', 'fontsize':'14'})
9 plt.title('Ubicacion de las diferentes marcas', fontsize= 32)
10 plt.show()
```

Datos personales

- **Apellidos:** Motino Flores
- **Nombre:** Skarleth
- **Email:** skarlethgm@gmail.com (<mailto:skarlethgm@gmail.com>)
- **Fecha:** 20 de Abril del 2023

Ficha de autoevaluación

En general considero la asignacion muy interesante, he comprendido mucho mejor el uso de diccionarios, con los cuales tenia muy poca experiencia.

Apartado	Calificación	Comentario
a)	3.0 / 3.0	Completamente resuelto
b)	2.0 / 2.0	Completamente resuelto
c)	1.0 / 1.0	Completamente resuelto
d)	3.0 / 3.0	Completamente resuelto
e)	0.5 / 0.5	Completamente resuelto
f)	0.3 / 0.5	Lo he conseguido más que mínimamente
Total	9.8 / 10.0	Aprobado

Ayuda recibida y fuentes utilizadas

- Documentacion de Numpy: <https://numpy.org/doc/stable/reference/generated/numpy.dtype.html> (<https://numpy.org/doc/stable/reference/generated/numpy.dtype.html>)
- Uso de bucles para Pandas <https://note.nkmmk.me/en/python-pandas-dataframe-for-iteration/> (<https://note.nkmmk.me/en/python-pandas-dataframe-for-iteration/>)
- Default values para argumentos de funciones. <https://realpython.com/python-optional-arguments/> (<https://realpython.com/python-optional-arguments/>)
- Documentacion de Geopandas: https://geopandas.org/en/stable/getting_started.html (https://geopandas.org/en/stable/getting_started.html)
- Libreria Geopandas: <https://github.com/joncutrer/geopandas-tutorial/tree/master/data> (<https://github.com/joncutrer/geopandas-tutorial/tree/master/data>)

Comentario adicional

Para realizar este proyecto use una Mac, en general creo debe ser similar a la sintaxis que en una computadora con Windows. La única diferencia que creo de ser considerada es cuando se usa Map Reduce, las direcciones de archivos van entre comillas, y no inclui "/" porque me daba error. También al usar el archivo para la función Map Reduce, decidí quitar la primera línea del archivo, ya que leía los encabezados de cada columna y me generaba error por el tipo de dato.

In [41]:

```
1 # Esta celda se ha de respetar: está aquí para comprobar el funcionamiento de algunas funciones por parte
```